Delta Bot: Complete Guide

What it is, how it works, how it was built, and how to deploy it

---

Version 1.3 · November 2026 · built by $um!t_K@Ktw@n · for Delta Exchange India perpetual futures (BTCUSD, ETHUSD, SOLUSD)
Project: github.com/Sumit211995/delta_bot

• Current mode: PAPER TRADING ONLY. No real orders.

> **Current mode: PAPER TRADING ONLY.** `DRY_RUN=1` is set in `.env`. The bot sends **no real orders**. It simulates every trade with the same entry, stop loss and target it would use with real money, and records the results. Real trading only starts if *you* change `DRY_RUN=0` **and** set `LIVE_CONFIRM=I_UNDERSTAND_THE_RISK`.

What changed in version 1.3 (13 Nov 2026)

Change	Why
Fix: a `.env` line like `ML_MODEL_PATH= # comment` was read by Docker as the file name `# comment`	On the server every real signal crashed, so the paper records stayed empty for ~2 days. `config.py` now treats a comment-only value as empty, and `.env.example` keeps comments on their own line.
1d trail skips very volatile days (ATR% in its top 20% of 500 candles)	Tested walk-forward: PF 1.57 → 1.69 and max drawdown 12.5R → 8.1R on unseen 2024–26 data (section 13.5).
BTC trend filter, funding-rate filter and a Donchian breakout strategy were tested	None improved results in both periods, so the 4h strategy is unchanged (section 13.5).
Dashboard on your phone through **Tailscale**	Private link, no open ports (section 16.5).

Contents

1. What the bot does, in one page
2. The big picture: parts and how they connect
3. One loop of the bot, step by step
4. The strategy in depth: why it buys or sells
5. Stop loss, target and position size, with a worked example
6. Safety layer: how the bot protects your money
7. Paper trading: every record (30m, 1h, 4h, 4h trail, 1d trail)
8. The dashboard
9. Telegram alerts
10. How we built it: the development story
11. Tools used and why
12. Why Python?
13. Test results: what the data says (incl. trailing-stop comparison)
14. Daily use: commands
15. Configuration reference (`.env`)
16. Deployment: running it 24/7 on a free server
17. Can I use the trailing-stop strategy? Going live later (not now)
18. Security rules
19. Troubleshooting and FAQ
20. Glossary

1. What the bot does, in one page

The bot watches the price of **BTC, ETH and SOL** on Delta Exchange India. Every time a **4-hour candle closes** (05:30, 09:30, 13:30, 17:30, 21:30 and 01:30 IST), it checks three fixed rules:

#	Rule	In plain words
1	**Trend**	Is the coin in an uptrend (EMA 50 above EMA 200, price above EMA 200) or a downtrend (the opposite)? In an uptrend it may only **buy**. In a downtrend it may only **sell**.
2	**Pullback**	Did the price dip against the trend? RSI fell **below 40** in an uptrend, or rose **above 60** in a downtrend.
3	**Trigger**	Has the dip ended? RSI just closed **back above 40** (buy) or **back below 60** (sell).

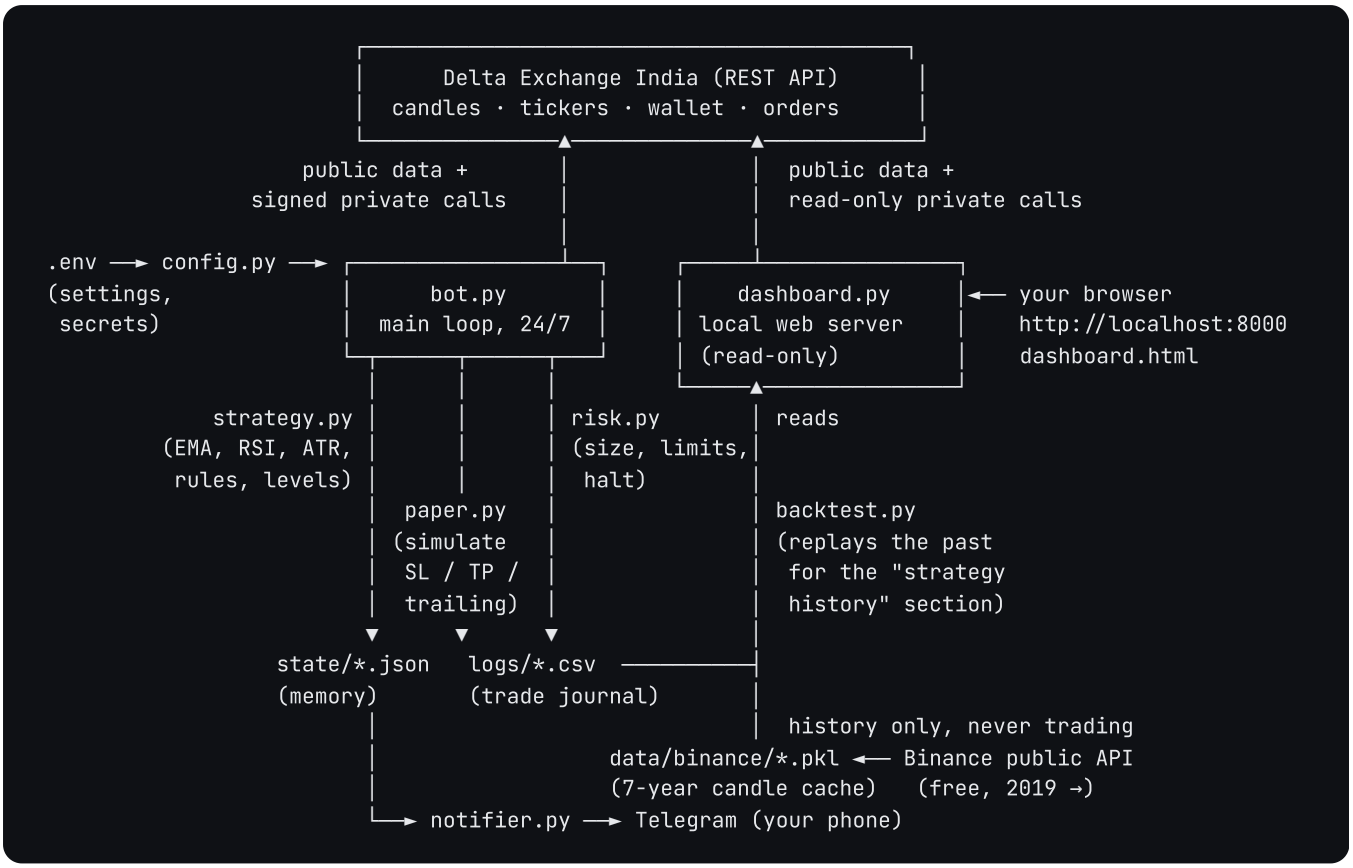
When all three are true, there is a **signal**. The bot then:

- sets a **stop loss** 1.5 × ATR away from the entry (ATR = how much the coin normally moves in one candle),
- sets a **target** 3 × the stop distance away (you risk 1 to make 3),
- picks a **size** so that hitting the stop loses only **0.5% of capital**,
- in **paper mode** (now): records the trade and follows it candle by candle until the stop or target is hit,
- in **live mode** (later): sends **one bracket order** to Delta, with the stop loss and target stored **on the exchange**.

Next to the main 4h record, the bot keeps **paper-only** records of the same rules on other settings: 30m, 1h, **4h trail** (trailing stop instead of a fixed target) and **1d trail** (daily candles). They show live, with real prices, which version works best before any real money is used.

It does **not** predict the future. It follows a fixed rule set that was tested on about 2.5 years of past data and showed a small edge. It wins only about **30%** of trades but makes money because each win is about 3× the size of each loss.

2. The big picture: parts and how they connect



Two programs, one set of files. `bot.py` is the only program that can trade. `dashboard.py` only **reads** the same `state/` and `logs/` files plus public prices, so the web page can never place an order. On the server each runs in its own Docker container, and both containers share the folders on the server's disk.

2.1 Tech stack at a glance

Layer	Technology	Where
Language	**Python 3.11** (bot, strategy, tests, web server) · HTML/CSS/JavaScript (dashboard page only)	all `*.py`, `dashboard.html`
Data and maths	**pandas**, **NumPy** (candles, EMA/RSI/ATR, fast backtests)	`strategy.py`, `backtest.py`
Exchange	**Delta Exchange India REST API v2**, signed with **HMAC-SHA256**, IPv4 forced for the IP whitelist	`delta_client.py`
Long history (research only)	**Binance** public futures candles, cached on disk	`dashboard.py`, `data/binance/`
Storage	Plain **JSON** (state) and **CSV** (trade journal) files, no database	`state/`, `logs/`
Web dashboard	Python's built-in `http.server` + **TradingView Lightweight Charts**, read-only, bound to localhost	`dashboard.py`, `dashboard.html`
Alerts	**Telegram Bot API**	`notifier.py`
Tests	**pytest**, 20 tests	`tests/`

Layer	Technology	Where
Packaging and hosting	**Docker + Docker Compose** on **Oracle Cloud Always Free** (Ubuntu, Mumbai/Hyderabad)	`Dockerfile`, `docker-compose.yml`
Source and updates	**Git + GitHub** (`Sumit211995/delta_bot`); the server pulls from GitHub	

File	Lines	Job
`bot.py`	~370	The main program. Runs forever, checks every 20 seconds whether a candle closed, and places or simulates trades for every record (4h, 30m, 1h, 4h trail, 1d trail).
`strategy.py`	~150	The "brain": indicators (EMA, RSI, ATR), the buy/sell rules, stop/target levels, and the trailing-stop rule.
`risk.py`	~80	Position size, daily loss limit, max drawdown halt, trades-per-day cap. Remembers state on disk.
`paper.py`	~45	Simulates paper trades candle by candle: fixed stop/target, or trailing stop. Same rules as the backtest.
`delta_client.py`	~145	Talks to Delta's API. Signs private requests with your secret key (HMAC-SHA256).
`config.py`	~160	Reads `.env`, applies safe defaults, refuses unsafe settings (e.g. trailing exit in live mode).
`notifier.py`	~15	Sends Telegram messages.
`dashboard.py` + `dashboard.html`	~610 + ~1,330	The live web screen (read-only; it can never place an order). Also downloads and caches the Binance long history.
`backtest.py`	~175	Replays history. Contains the fast simulator used by the dashboard's strategy history.
`train_model.py`	~60	Optional machine-learning filter (off by default; not needed).
`tests/`	~300	20 automatic tests: core logic, `.env` parsing, the volatility filter.
`research/addon_filters.py`	~250	Walk-forward test of popular add-ons (BTC regime, funding rate, ATR filter, Donchian).
`docs/GUIDE.md`, `docs/build_pdf.py`		This guide and the script that turns it into the PDF.
`Dockerfile`, `docker-compose.yml`		Packaging for running on a server.

3. One loop of the bot, step by step

The bot is a simple loop. Every **20 seconds** it does this for each record (4h, then the paper records 30m, 1h, 4h trail and 1d trail) and each coin:

```
1. Download the last ~400 candles for the coin        (public API, no key needed)
2. Drop the candle that is still forming              (only CLOSED candles are used)
3. Newest closed candle already handled?   ──yes──►  nothing to do, next coin
        │ no
4. Save "handled" to state/ FIRST                     (a crash can never double-trade)
5. Paper: open trade hit its SL or TP?     ──yes──►  close it, update balance, Telegram
6. Compute EMA 50, EMA 200, RSI 14, ATR 14
7. Check the 3 rules on the closed candle  ──no───►  log "no signal", next coin
        │ signal
8. Position already open on this coin?     ──yes──►  ignore the signal
9. Halted? daily loss limit? trade cap?    ──fail──► Telegram "skipped: reason"
10. Entry = mark price; SL/TP from ATR; size from risk %
11. Size < 1 contract?                     ──yes──►  Telegram "skipped: too small"
12. PAPER: record position + log + Telegram
    LIVE : one bracket order (market + SL + TP), then confirm SL/TP exist on Delta
           if not confirmed: close the position, HALT, Telegram alert
```

Why only closed candles? A candle that is still forming can "flash" a signal that disappears before it closes. The backtest only used closed candles, so live trading must too. Otherwise the live results would not match the test. That is why there is no new signal **between** the 4h closes. The dashboard shows a dashed amber **"forming"** hint if a signal *might* appear at the next close, but it only counts once the candle closes.

Why "mark handled" before ordering? If the bot crashed between sending an order and saving, it could restart and send the same order again. Marking first means the worst case is one *missed* trade, never a *duplicate* one. For the same reason, order requests are **never retried automatically** (only read requests are).

4. The strategy in depth: why it buys or sells

The strategy is called **trend pullback**. The idea is old and simple: *trade in the direction of the big trend, but enter after a temporary dip, not at the top of a move.*

4.1 The indicators

Indicator	Formula (simplified)	What it tells us
EMA 50	Exponential moving average of the last ~50 closes	Medium-term direction
EMA 200	Exponential moving average of the last ~200 closes	Long-term direction (the "big trend")
RSI 14	100 − 100 / (1 + average gain / average loss), over 14 candles (Wilder smoothing)	Momentum, 0–100. Low = recently sold off, high = recently pumped
ATR 14	Average of the "true range" (high − low, including gaps) over 14 candles	How much the coin normally moves in one candle, in dollars

On 4h candles, EMA 200 covers about **33 days** and EMA 50 about **8 days**.

4.2 The rules (exact code in `strategy.py → _pullback`)

```
uptrend   = EMA50 > EMA200  and  close > EMA200
downtrend = EMA50 < EMA200  and  close < EMA200

BUY  if uptrend   and previous RSI < 40  and  current RSI ≥ 40     (dip is over, trend resumes up)
SELL if downtrend and previous RSI > 60  and  current RSI ≤ 60     (bounce is over, trend resumes
down)
```

4.3 Why this works (when it works)

- **Trend filter:** markets tend to keep going in the direction of their long-term trend more often than not. Trading against it means fighting the bigger players.
- **Pullback:** buying after a dip gives a better price than buying a breakout, so the stop loss can sit just below normal noise.
- **Trigger on the cross back:** waiting for RSI to turn back avoids "catching a falling knife". The bot enters only once the dip shows signs of ending.
- **3:1 reward:** most trades lose (about 70%), but each winner pays for about three losers. You need a win rate above 25% plus fees to make money; the test showed about 30%.

4.4 Why it does not "predict"

There is no forecast, no AI guess and no news reading. The same candles always give the same answer. That is a strength: you can test it on years of history and know exactly why every trade happened. (An optional ML filter exists in `train_model.py`, but it did not improve results, so it is off.)

4.5 Two ways to exit a trade

	Fixed 1:3 (the 4h record; used for live orders)	**Trailing stop** (the "4h trail" paper record)
Stop loss at entry	1.5 × ATR	1.5 × ATR (same)
Target	3 × the stop distance	none
After +1R profit	nothing changes	the stop follows 2 × ATR behind the best close, only moving in your favour
Trade ends when	the stop or the target is hit	the (moved) stop is hit
Typical result	−1R or +3R	−1R, small win, or now and then a large win (+5R to +17R)

Both use exactly the same entry signals. Section 13 compares them.

5. Stop loss, target and position size, with a worked example

```
stop distance = 1.5 × ATR
BUY : stop loss = entry - stop distance      target = entry + 3 × stop distance
SELL: stop loss = entry + stop distance      target = entry - 3 × stop distance

size (contracts) = floor( capital × risk% ÷ (stop distance × contract value) )
                   also capped so that  size × price × contract value ≤ capital × max leverage
```

Example: ETH sell signal on the 4h chart (paper capital $500)

Step	Value
Entry (mark price)	2,104.00
ATR (4h)	34.70
Stop distance = 1.5 × 34.70	52.05
Stop loss = 2,104 + 52.05	**2,156.05**
Target = 2,104 − 3 × 52.05	**1,947.85**
Contract value (ETHUSD)	0.01 ETH
Loss per contract if stopped = 52.05 × 0.01	$0.52
Risk budget = $500 × 0.5%	$2.50
Size = floor(2.50 ÷ 0.52)	**4 contracts** (0.04 ETH)
Leverage cap = floor(500 × 2 ÷ (2,104 × 0.01))	47 contracts, so it does not limit here
Money at risk if stopped	about **$2.08**
Money made if target hit	about **$6.25**
Fees (0.05% taker + 18% GST, both sides)	about $0.10

Contract sizes on Delta India: BTCUSD = 0.001 BTC, ETHUSD = 0.01 ETH, SOLUSD = 1 SOL. With a very small wallet, some coins cannot be traded at all. For example, one SOL contract risks about $2.40 at a normal stop, which is more than 2% of a $27 wallet. The bot then **skips** the signal instead of over-risking.

6. Safety layer: how the bot protects your money

Protection	Default	What happens
Paper mode by default	`DRY_RUN=1`	No real orders until you change two separate settings.
Live confirmation phrase	`LIVE_CONFIRM`	Live trading refuses to start unless it equals `I_UNDERSTAND_THE_RISK`.
Risk per trade	0.5% (max allowed 2%)	Size is calculated from the stop distance; the config refuses more than 2%.
Capital cap	`BOT_CAPITAL_USD`	The bot sizes from the smaller of the wallet and this cap, even if the wallet is bigger.
Leverage cap	2× (max 10×)	Position value is limited to capital × leverage.
Daily loss limit	−3%	No new trades until the next UTC day.
Max drawdown halt	−10% from peak	The bot **stops permanently** until you review it and reset by hand.
Trades per day	3	Cap per timeframe record.
One position per coin		New signals on a coin are ignored while it has an open position (including your manual trades).
Stop loss on the exchange		Live orders are *bracket* orders: the SL and TP live on Delta's servers, so they work even if your laptop is off.
Protection check		After a live entry, the bot checks that new SL and TP orders exist. If not, it closes the position and halts.
No duplicate orders		Candle marked "handled" before ordering; order requests are never retried.
Fail-safe on errors		If the wallet can't be read, no trade. 20 errors in a row → exit (Docker restarts it).
Panic button	`python bot.py --panic`	Cancels all orders and closes positions on the bot's coins (manual ones too, so be careful).
Read-only dashboard		The web page has no buttons that trade. It only listens on your own machine (127.0.0.1).

7. Paper trading: every record (30m, 1h, 4h, 4h trail, 1d trail)

The strategy was designed and tested for **4h**. On 30m and 1h the backtest **lost money** (none of 91 setting combinations made money after fees). They are still paper-traded side by side so you can **see this for yourself** with live data before trusting any timeframe with real money.

Timeframe	Checked	Mode	Record files
4h ★	every 4 hours	paper now (live later if you choose)	`state/paper_state_4h.json`, `logs/paper_trades_4h.csv`
1h	every hour	**paper only, always**	`state/paper_state_1h.json`, `logs/paper_trades_1h.csv`
30m	every 30 minutes	**paper only, always**	`state/paper_state_30m.json`, `logs/paper_trades_30m.csv`
4h trail	every 4 hours	**paper only, always**: same signals, trailing stop instead of a fixed target	`state/paper_state_4h-trail.json`, `logs/paper_trades_4h-trail.csv`
1d trail	once a day (05:30 IST)	**paper only, always**: daily candles with the trailing stop; skips signals on very volatile days (top 20% ATR%)	`state/paper_state_1d-trail.json`, `logs/paper_trades_1d-trail.csv`

- Each timeframe starts with its own **$500** paper balance (`BOT_CAPITAL_USD`) and its own limits.
- `PAPER_TIMEFRAMES=30m,1h` in `.env` controls the extra timeframes. Even when 4h goes live, these stay paper-only.
- **How a paper trade closes:** after each closed candle, the bot checks whether the candle's high/low touched the stop or target. If **both** were touched in the same candle, it assumes the **stop** came first (the pessimistic choice, same as the backtest).
- **P&L includes fees:** 0.06% of the trade value on entry and on exit (0.05% Delta taker fee + 18% GST).
- **Entry price** = the mark price at the moment the candle closed, which is close to what a real market order would get.

7.1 The trailing-stop test (4h trail)

Long-history testing (Binance, 2019–2026, 10 coins, plus Delta 2023–2026) found one exit that clearly beat the fixed 1:3 target: **a trailing stop**. The "4h trail" record takes exactly the same signals as 4h, but:

- **Stop loss at entry:** the same 1.5 × ATR.
- **Target:** none.
- **Once the trade is +1R in profit:** after each closed 4h candle, the stop moves to 2 × ATR behind the best close so far. It only ever moves in the trade's favour.
- **Exit:** when the stop is hit. The journal shows **✕ STOP** (the initial stop, −1R) or **✓/✕ TRAIL STOP** (the moved stop).

BTC+ETH+SOL, 4h	Fixed 1:3	Trailing stop
Delta 2023–2026	+38R · win 30%	**+112R · win 40%**

BTC+ETH+SOL, 4h	Fixed 1:3	Trailing stop
Binance 2019–2026	+102R · worst streak 21	**+195R · worst streak 11**

It is **paper only**: real orders still use the fixed SL + TP bracket, and the config refuses the trailing exit in live mode. Moving a real stop order on Delta after every candle needs extra live code and testing. Compare the two 4h records for a few weeks first.

8. The dashboard

Start it with `python dashboard.py` and open **http://localhost:8000**.

Section	What it shows
Top bar	Mode (PAPER / LIVE), connection status, alerts button 🔕/🔔, light/dark switch
Price strip	Live price of each coin (updates every 2 s, flashes green/red), 24h high/low, funding, volume
Signal cards (one per coin)	**NO SIGNAL** (grey, radar animation, what it is waiting for, countdown) · **▲ BUY SIGNAL / ▼ SELL SIGNAL** (pops in, glows, shows entry / SL / target / size / risk → reward) · **IN TRADE** (live P&L)
Chart	Candles with EMA 50 / EMA 200, **B** (buy) and **S** (sell) markers for every past signal, SL / target / entry lines for open trades, RSI pane with the 40/60 levels. History: 1M, 3M, 6M, 1Y, 2Y, ALL. Timeframes: 30m, 1h, 4h. The last candle moves live.
Signal ticket + rule checklist	The three rules with ✓ / … / ✕, and a ring counting down to the next candle close
Signals across timeframes	A tile per coin per timeframe: NO SIGNAL / BUY / SELL, RSI meter, trend, time to next close
Paper trading · 30m vs 1h vs 4h	A score card per timeframe (balance, P&L, win rate, W/L, average R, max drawdown, open trades) and a **trade journal**: one row per trade with timeframe, coin, side, opened time, **entry, stop loss, target**, status (OPEN / ✓ TARGET / ✕ STOP), exit, closed time, time held, P&L and R. Filters for timeframe / coin / status, and **Download CSV**.
Account & risk	Mode, Delta wallet, risk settings
Data source switch (strategy history)	**Delta (≈2.5 years)** or **Long history · Binance (2019 →)**. The Binance option replays the same rules on ~7 years of Binance perpetual candles (free public API). It is downloaded once (2–5 min, saved in `data/binance/`, git-ignored) and then only topped up with new candles. It is for **research only**: the bot always trades on Delta's prices. Open it directly with `http://localhost:8000/?hsrc=binance`.
Strategy comparison (top of the strategy history)	One line per method (30m, 1h, 4h fixed, 4h trail) with all coins combined: trades, trades per month, **success (win rate)**, profit factor, **net profit in $**, return, total R, **₹ result at ₹100 risk per trade**, worst losing streak, max drawdown and best trade. The best value in each column is green.
Loading shimmer	Every section shows a moving grey placeholder until its data arrives (prices, signals, chart, journal, history).
Strategy history · every past trade (bottom of the page)	The strategy replayed on **all available Delta data** (about 2.5 years) for every coin and every timeframe: a score card per coin × timeframe (trades, win rate, return, profit factor, max drawdown) and a table of **every past trade** with entry, stop loss, target, result, exit, time held, P&L, R and balance. Filters for coin / timeframe / result / side / year, 50 trades per page, **Download CSV**, and **click any row to see that trade on the chart** with its entry/SL/target lines. Simulated with fees and slippage on a separate $500 balance per coin/timeframe.
Open trades / Event log / Delta orders	Open positions with live P&L, the raw bot log, and your real open orders on Delta (read-only)

Other features: `?demo` in the URL shows an example signal (nothing is traded), desktop notifications and a flashing tab title on new signals, phone-friendly layout, all times in IST.

How it gets data: the page asks the local Python server every **2 seconds** for prices and the forming candle (`/api/live`) and every **10 seconds** for signals and paper records (`/api/overview`). The server fetches public data from Delta and caches it briefly so Delta is not flooded. Charts use up to 2 years of candles, fetched in 1,000-candle pieces in parallel and cached for 10 minutes.

9. Telegram alerts

The bot sends a message for: bot started/stopped, every new trade (with timeframe, entry, SL, TP, risk), every stop or target hit (with P&L and new balance), signals that were skipped and why, and errors.

Example: `PAPER 1h SELL 1 SOLUSD @~86.60 SL 88.40 TP 81.20 (risk ~1.80) | downtrend pullback: RSI back below 60`

Setup (already done for you):

1. In Telegram, open **@BotFather** → `/newbot` → choose a name and a username ending in `bot` → copy the **token** into `TELEGRAM_TOKEN=` in `.env`.
2. Open your new bot, tap **Start**, send "hi".
3. Run the chat-id helper from the README (it reads the token from `.env` without printing it) and put the number into `TELEGRAM_CHAT_ID=`.
4. Restart the bot. You should get "Bot started ..." on your phone.

The token only controls the Telegram bot, not your Delta account. Still keep it private.

10. How we built it: the development story

This bot was built step by step in about one working session, using **Claude Code** (an AI coding assistant running in VS Code) together with you. The steps were:

Step	What was done	Why
1. Review	Read the original code: a single-coin EMA-cross bot on 1h	Understand what exists before changing anything
2. Research the API	Studied Delta Exchange India's API: signing, bracket orders, candles, India base URLs, IP whitelist	Orders must be correct the first time. Real money is involved.
3. Honest backtest	Built a fast test engine: entry on the **next** candle's open, stop checked before target, fees + GST + slippage	A backtest that cheats (look-ahead, no fees) gives false hope
4. Found the old strategy loses	EMA-cross on 1h: about −36% (BTC) and −47% (ETH)	Better to know before trading it live
5. Search for a better rule set	Tried 193 combinations (EMA cross, breakouts, pullbacks; different stops and targets) on the **first 60%** of the data only	Tuning on all the data would just "memorise" the past
6. Walk-forward check	Tested the winner on the **last 40%**, which it had never seen	The only honest way to check that an edge is real
7. Picked trend pullback on 4h	Profitable on unseen data for both BTC and ETH	Small but real edge; simple rules
8. Tested 30m and 1h	0 of 91 combinations profitable after fees	Fees eat small-timeframe edges, so these became watch-only
9. Safety layer	Risk manager, halts, protection check, no-duplicate design, panic, live confirmation phrase	A bot must fail safe
10. Multi-coin + paper trading	BTC + ETH (later SOL), paper mode with simulated SL/TP and fees	Practise with no money
11. Dashboard	Read-only web page: live prices, chart, signals, rule checklist; then a professional redesign with animations	See what the bot sees, in real time
12. Real bug found and fixed	Delta's `/v2/orders` **ignores** `product_id` and returns every coin's orders, so BTC showed ETH's orders. Fixed with `product_ids` plus a second check on our side, and a test.	This could have fooled the live "is my stop loss there?" check
13. Added coins carefully	Tested 14 popular coins; only SOL was both profitable and liquid enough	Adding losing coins makes the bot worse
14. Paper on all timeframes + journal	One bot paper-trades 30m/1h/4h; the dashboard journal shows every trade	Compare timeframes with live data
15. Tests + Git	Automated tests for every important rule; every change committed to Git; pushed to GitHub `Sumit211995/delta_bot`	Prove the logic works and keep history

Step	What was done	Why
16. Guide + PDF	This guide, written in Markdown and printed to PDF with headless Chrome	One place that explains everything
17. Strategy history	Replayed the strategy on all of Delta's history (about 2.5 years) for every coin and timeframe and showed each past trade with entry, SL, target and result	See exactly what the rules would have done
18. Liquidation research	Delta has no liquidation data, so we tested free stand-ins: funding rate (crowded longs/shorts) and open-interest drops (position flushes)	All three filters cut trades and profit, so none was added
19. "More trades" research	630 variants: shallower pullbacks, looser trend, 2h, 1h, 4h-trend + 1h-entry, other stop/target sizes; tuned on the first 60%, judged on the last 40%	None beat the current rules on unseen data. More trades meant worse trades.
20. Long-history test	Downloaded free Binance perpetual history (2019–2026, 10 coins) and tested exits and filters on 6–7 years	The current strategy held up (PF 1.22 on BTC+ETH+SOL). One exit clearly beat it: the trailing stop.
21. Robustness check	Tried 16 trailing settings, older vs newer years, every coin, and removed the 5 biggest wins	The trailing stop still won in almost every case
22. Trailing-stop paper record	Added "4h trail" as its own paper record (same signals, no fixed target), with tests proving the live paper logic exits exactly like the backtest, plus the comparison table and loading shimmer on the dashboard	Compare both exits on live prices before any real money uses it

11. Tools used and why

Tool	Used for	Why this one
Python 3	The bot, strategy, backtests, dashboard server, tests	See section 12
pandas	Candle tables, indicators (EMA, RSI, ATR)	The standard tool for time-series data; one line per indicator
NumPy	Fast maths, vectorised signal search over years of data	Hundreds of times faster than plain loops
requests	HTTP calls to Delta and Telegram	Simple, reliable, the most used HTTP library in Python
hmac / hashlib (built in)	Signing private API requests (HMAC-SHA256)	Required by Delta; built into Python, no extra package
http.server (built in)	The dashboard's local web server	No framework needed for a small read-only page; nothing extra to install or secure
HTML + CSS + JavaScript	The dashboard page	Browsers only run JavaScript, so the page itself has to use it. No build step and no framework, just one file.
TradingView Lightweight Charts (open source)	The candle chart with markers and price lines	Fast with years of candles, looks professional, free
Inter / JetBrains Mono (Google Fonts)	Text and numbers on the dashboard	Clean, easy-to-read numbers
pytest	Automatic tests	Python's standard test runner
Docker + Docker Compose	Running on a server	Same environment everywhere; automatic restart; one command to start
Git + GitHub	Version history, backup, deployment source	Every change is recorded and reversible; the server pulls code from GitHub
Telegram Bot API	Phone alerts	Free, instant, no app to build
Delta Exchange India API v2	Market data and orders	The exchange you trade on; public data needs no key
Binance public API (history only)	2019–2026 perpetual candles for the long-history research and the dashboard's "Long history" data source	Free, no account, ~7 years instead of 2.5. Only public prices are downloaded; signals and trades always use Delta's prices.
VS Code + Claude Code	Editing; AI pair-programming (code, research, tests, review, screenshots)	Faster development with a second pair of eyes on safety
Headless Google Chrome	Screenshots to check the UI; turning this guide into a PDF	Already on the Mac; renders exactly like the browser

Tool	Used for	Why this one
Oracle Cloud Always Free (planned)	24/7 hosting	Free forever VM in India with a fixed public IP

Not used, on purpose: no paid data feeds, no paid libraries, no database server (JSON and CSV files are enough and easy to inspect), and no heavy web framework.

12. Why Python?

The bot itself, the strategy, the tests and the dashboard server are all **Python**. Only the dashboard page uses HTML/JavaScript, because that is the only language a web browser runs. The reasons:

1. **The backtest and the live bot use the same code.** `strategy.py` is imported by both `backtest.py` and `bot.py`. The rules that were tested are exactly the rules that trade. In a two-language setup (e.g. research in Python, trading in Java) the two versions slowly drift apart, and the live results stop matching the test.
2. **The best tools for market data.** pandas and NumPy make indicators one line each and test years of data in seconds. Almost every trading tutorial, exchange example and quant library is in Python.
3. **Speed is not the problem here.** The bot acts once per candle (every 4 hours on the main timeframe) and spends most of its time waiting. Python is more than fast enough. C++/Rust/Go only matter for high-frequency trading in milliseconds, which this is not.
4. **Easy to read and change.** The code reads almost like English, so you can open `strategy.py` and see the exact rules.
5. **Few moving parts.** Three main packages (requests, pandas, numpy). Fewer dependencies mean fewer things to break or attack.
6. **Easy to deploy.** One Docker image based on the official `python:3.11-slim` runs on any Linux server, including the free Oracle VM.

Alternatives and why not: Node.js (good for web, weaker for data analysis), Java/C# (more code for the same thing), Go/Rust (fast, but the speed is not needed and the data tools are weaker), no-code platforms like Tradetron/AlgoTest (monthly fees, less control, and you would need to whitelist their servers on your API key).

13. Test results: what the data says

All tests include fees (0.06% per side), slippage (0.02%) and entry on the **next** candle. Numbers are for the **4h trend-pullback** strategy, 1% risk per trade, Delta India data from late 2023 / early 2024 to November 2026.

Main coins (README backtest, settings tuned on the first 60%, checked on the last 40%):

Coin	Trades	Win rate	Profit factor	Return	Max drawdown
BTCUSD	99	30%	1.17	+14.0%	9.2%
ETHUSD	88	31%	1.23	+15.8%	11.0%

14 popular coins (same settings, never tuned on them):

Coin	Full period PF	Last 40% PF	Decision
ETH	1.30	1.30	✅ traded
SOL	1.13	1.48	✅ traded (added)
BTC	1.18	1.06	✅ traded (weaker lately)
DOGE, BCH	1.29 / 1.54	positive	⚠ not added: only ~$2M daily volume on Delta (bad fills)
BNB, AVAX, TRX	about 1.0	mixed	❌ no edge
XRP, ADA, LINK, DOT, SUI, LTC	below 1.0		❌ lose money

Timeframes: 30m and 1h: **0 of 91** setting combinations were profitable on unseen data after fees. 4h: robust.

How to read this: profit factor (PF) = money won ÷ money lost; above 1.0 is profitable. **1.1–1.3 is a small edge.** Expect losing months. About **7–8 trades per month** across BTC + ETH + SOL on 4h. Past results do not guarantee future results.

13.1 Strategy comparison: all coins combined (Delta data, Dec 2023 – Nov 2026)

Each coin starts with $500 (so $1,500 in total), 0.5% risk per trade, compounding, fees + slippage included. This is the same table the dashboard shows at the top of the strategy history.

Method	Trades	Per month	Success (win rate)	Profit factor	Net profit	Return	Total R	At ₹100 risk / trade	Worst losing streak	Max drawdown
30m fixed 1:3	2,008	58	26.1%	0.87	−$484	−32.2%	−228R	−₹22,813	25	49.9%
1h fixed 1:3	996	29	25.9%	0.90	−$188	−12.5%	−73R	−₹7,276	28	22.5%

Method	Trades	Per month	Success (win rate)	Profit factor	Net profit	Return	Total R	At ₹100 risk / trade	Worst losing streak	Max drawdown
4h fixed 1:3 (current)	257	7.5	30.0%	1.19	**+$93**	+6.2%	+38R	**+₹3,827**	20	5.8%
4h trailing stop	286	8.3	**39.9%**	**1.68**	**+$293**	**+19.5%**	**+112R**	**+₹11,214**	**11**	**5.5%**

"At ₹100 risk / trade" means: if every trade risked exactly ₹100, this is what all the trades together would have made.

13.2 Long history: Binance 2019–2026 (BTC + ETH + SOL, 4h)

Exit method	Trades	Win rate	Profit factor	Total R	Worst losing streak
Trailing stop (after +1R, 2 × ATR)	733	**41.7%**	**1.48**	**+195R**	**11**
Stop to breakeven at +1R	713	24.5%	1.31	+121R	
Fixed 1:3 (current)	635	30.1%	1.22	+102R	21
Half profit at 1.5R	695	43.9%	1.19	+77R	
Fixed 1:2	683	36.3%	1.07	+32R	

On all 10 Binance coins the current strategy was profitable on 7 (not LINK, LTC or XRP). The trailing stop beat the fixed target in **both** 2019–2022 and 2023–2026, with 16 different trailing settings. It still won after removing its 5 biggest trades.

13.3 Daily (1D) candles

BTC + ETH + SOL	Trades / month	Win rate	Profit factor	Total R
1D fixed 1:3 · Binance 2019–2026	1.1	30%	1.25	+16.5R (lost money 2023–26)
1D trailing · Binance 2019–2026	1.3	**45%**	**2.15**	**+70.8R** (profitable in both 2019–22 and 2023–26)
1D fixed 1:3 · Delta 2024–2026	1.2	19%	0.69	−10.8R
1D trailing · Delta 2024–2026	1.4	37%	1.30	+9.5R (only 49 trades)
4h trailing · Binance, for comparison	8.5	42%	1.48	+195R

Daily candles with the **trailing stop** are high quality but rare (about one trade a month across the 3 coins), so "1d trail" runs as a paper record. Daily with the fixed target is not worth trading.

Daily trend as a filter for 4h (only buy on 4h when the daily chart is also up): tested strict and loose versions. It **cut profit in every case** (e.g. 4h trail +195R → +118R to +125R) and is not used.

13.4 What did not help (tested and rejected)

Idea	Result on unseen data
Liquidation stand-ins: skip trades when funding shows crowded longs/shorts	Trades halved, profit cut by more than half
Liquidation stand-in: trade only after a big open-interest drop ("flush")	Worse, and SOL lost money
Shallower pullback (RSI 45/55) for more trades	Lost money (PF 0.91)
2h or 1h candles, or 4h trend + 1h entry	2–6× more trades, all about break-even or losing
Target 1:2 instead of 1:3	About half the long-run profit
ADX trend-strength filter	Fewer trades, no improvement
Daily trend filter on 4h trades	Fewer trades, less profit in every version
Daily candles with the fixed 1:3 target	Lost money on Delta and on Binance since 2023

13.5 Popular add-ons, tested Nov 2026 (`research/addon_filters.py`)

Method: Binance BTC + ETH + SOL. Every setting was chosen on **2019–2023 only**, then checked on **2024–2026** (untouched). An add-on is kept only if it raises profit factor in **both** periods, keeps ≥ 60% of the trades and does not raise the drawdown. Each idea had at most 6 settings, so luck has little room.

Idea (best setting on 2019–23)	Book	PF 2019–23	PF 2024–26	Max DD 2024–26	Kept?
Baseline pullback	4h	1.34	1.09	29.9R	—
BTC above its EMA200 before trading ETH/SOL	4h	1.44	1.05	22.4R	No
Funding-rate extremes (skip crowded side)	4h	1.53	1.08	23.5R	No
Skip top-20% volatility	4h	1.48	1.08	19.5R	No
Baseline pullback	4h trail	1.48	1.47	23.9R	—
BTC regime (daily EMA200)	4h trail	1.54	1.41	25.9R	No
Funding-rate extremes	4h trail	1.63	1.49	18.9R	No: +0.02 PF is noise and total profit fell (84.6R → 71.3R)

Idea (best setting on 2019–23)	Book	PF 2019–23	PF 2024–26	Max DD 2024–26	Kept?
Skip top-20% volatility	4h trail	1.60	1.58	20.8R	No: drawdown rose in 2019–23 (12.1R → 15.7R)
Baseline pullback	1d trail	3.22	1.57	12.5R	—
BTC regime (EMA200)	1d trail	3.84	1.67	11.2R	No: the volatility filter alone was better on 2019–23
Funding-rate extremes	1d trail	3.62	1.44	11.5R	No
Skip top-20% volatility (500 candles)	**1d trail**	**4.28**	**1.69**	**8.1R**	**Yes** (`VOL_FILTER_BOOKS=1d-trail`)
Donchian 20-bar breakout, ATR stop, 1:2 (standalone)	4h	1.29	1.09	51.1R	No: drawdown twice the pullback
Donchian 20-bar breakout (standalone)	1d	2.67	1.64	9.9R	No: no better than 1d-trail pullback with the filter

Honest note: the 1d-trail record has about 45–55 trades per period, so the gain is small and could be luck. It is a paper record only; the live 4h book is unchanged because nothing improved it in both periods.

14. Daily use: commands

Run these in the project folder (`cd ~/Documents/Project/delta_bot`):

Command	What it does
`caffeinate -i python bot.py`	Start the bot (4h + paper 30m + paper 1h). `caffeinate` stops the Mac sleeping. Stop with Ctrl+C.
`python dashboard.py`	Start the dashboard → open http://localhost:8000
`python bot.py --status`	Paper balance, open trades, wins/losses for every timeframe
`python bot.py --check`	Test the connection: product info, wallet balance, positions, open orders. Places no orders.
`python bot.py --once`	Process the latest closed candle once and exit
`python bot.py --paper-tf none`	Run only the main timeframe (no extra paper timeframes)
`python bot.py --symbols ETHUSD`	Override the coins for this run
`python backtest.py --symbol ETHUSD --resolution 4h`	Re-run the backtest with a per-year breakdown
`python -m pytest -q tests`	Run the automatic tests (should say "20 passed")
`python bot.py --panic`	**Emergency:** cancel all orders and close positions on the bot's coins (manual trades too)

15. Configuration reference (`.env`)

`.env` holds your settings **and secrets**. It is listed in `.gitignore` and must **never** be committed or shared. `.env.example` is the safe template.

Setting	Current / default	Meaning
`DELTA_ENV`	`live`	`live` = api.india.delta.exchange · `testnet` = fake-money test exchange
`DRY_RUN`	`1`	**1 = paper trading (no orders).** 0 = real orders
`LIVE_CONFIRM`	empty	Must be `I_UNDERSTAND_THE_RISK` for real orders on `live`
`DELTA_API_KEY` / `DELTA_API_SECRET`	set	Your Delta API key (Read Data + Trading; **never** Withdrawal)
`SYMBOLS`	`BTCUSD,ETHUSD,SOLUSD`	Coins to trade, comma separated
`RESOLUTION`	`4h`	Main timeframe (the tested one)
`PAPER_TIMEFRAMES`	`30m,1h`	Extra timeframes, paper only
`PAPER_TRAIL_TIMEFRAMES`	`4h,1d`	Also paper-trade with a trailing stop (no fixed target)
`TRAIL_ACTIVATE_R` / `TRAIL_ATR`	`1.0` / `2.0`	Trailing starts at +1R, stop 2 × ATR behind the best close
`STRATEGY`	`pullback`	`pullback` (default) or `ema_cross` (old, loses money)
`RSI_PULLBACK`	`40`	Buy level (sell level = 100 − this = 60)
`ATR_STOP_MULT`	`1.5`	Stop distance in ATRs
`REWARD_RISK`	`3.0`	Target = this × stop distance
`BOT_CAPITAL_USD`	`500`	Capital the bot sizes from (and paper starting balance)
`RISK_PER_TRADE`	`0.005`	0.5% risked per trade (max 0.02)
`MAX_LEVERAGE`	`2`	Leverage cap (1–10)
`DAILY_LOSS_LIMIT`	`0.03`	Stop new trades for the day at −3%
`MAX_DRAWDOWN`	`0.10`	Halt permanently at −10% from peak
`MAX_TRADES_PER_DAY`	`3`	Per timeframe record
`FEE_RATE`	`0.0006`	Per side, used for paper P&L and backtests
`TELEGRAM_TOKEN` / `TELEGRAM_CHAT_ID`	set	Phone alerts

Setting	Current / default	Meaning
`VOL_FILTER_BOOKS` / `VOL_FILTER_N` / `VOL_FILTER_MAX`	`1d-trail` / `500` / `0.8`	Skip signals when volatility is in its top 20% (see 13.5); `none` = off

16. Deployment: running it 24/7 on a free server

Right now the bot runs on your laptop, so it only works while the laptop is on, awake and online. A server runs it 24/7. **Deploy in paper mode first (`DRY_RUN=1`)** and keep it that way until you decide to go live.

16.1 What the server needs

- **Always on**, with internet
- A **fixed public IP**: Delta API keys only work from whitelisted IPs, and since April 2026 Indian brokers also require a static IP for API trading
- **Persistent disk** for `state/`, `logs/` (the bot's memory and trade journal) and `data/` (Binance history cache, ~50 MB)
- About **150 MB RAM** for both containers. Any small VM is enough.

16.2 Options

Option	Cost	Fixed IP	Verdict
Oracle Cloud Always Free (Mumbai / Hyderabad)	Free	Yes	**Recommended**
Your laptop	Free	No (mobile hotspot IP changes)	Fine for paper testing only
Paid VPS (AWS Lightsail Mumbai, DigitalOcean Bangalore, etc.)	about $4–6 / month	Yes	Good if Oracle signup fails
Railway / Render / Heroku-style platforms	about $5+ / month	Usually not	Not suitable (no fixed outbound IP on cheap plans)

16.3 Before you start (on your laptop)

1. Push the latest code to GitHub: `git push -u origin main` (use your Sumit211995 personal access token as the password).
2. Make sure `.env` is **not** on GitHub (it is in `.gitignore`). You will copy it to the server separately.
3. Keep `DRY_RUN=1` in the `.env` you copy.
4. Decide whether the server should **continue your laptop's paper records** (copy `state/` and `logs/` in Step 7) or start fresh.

16.4 Step by step: Oracle Cloud Always Free

Step 1: Create the account

1. Go to **cloud.oracle.com** → *Sign up*. A card is needed for identity verification; Always Free resources are not charged.
2. Choose home region **India West (Mumbai)** or **India South (Hyderabad)**. It cannot be changed later.

Step 2: Create the virtual machine

1. Menu → *Compute* → *Instances* → **Create instance**.
2. Image: **Canonical Ubuntu 22.04** (or 24.04).

3. Shape: **VM.Standard.E2.1.Micro** (AMD, 1 GB RAM) or **VM.Standard.A1.Flex** (Ampere ARM, 1 OCPU / 6 GB). Both are marked "Always Free eligible". If Ampere says "out of capacity", use E2.1.Micro or try again later.
4. Networking: keep the default VCN, **assign a public IPv4 address**.
5. SSH keys: **Generate a key pair** and **download the private key** (e.g. `oracle.key`). Keep it safe.
6. Click **Create** and wait until the instance is *Running*. Copy the **Public IP address**.
7. Optional but recommended: in the instance's VNIC → IPv4 addresses, change the public IP to a **reserved** IP so it never changes, even if the instance is recreated.

Step 3: Whitelist the server IP on Delta

1. Delta Exchange India → *Account* → **API Keys** → edit your key (or create a new one just for the server).
2. Add the server's public IP to the **whitelisted IPs**. Permissions: **Read Data + Trading** only. **Never enable Withdrawal.**

Step 4: Connect to the server (from your Mac terminal)

```
chmod 400 ~/Downloads/oracle.key
ssh -i ~/Downloads/oracle.key ubuntu@<SERVER_IP>
```

Step 5: Install Docker, Git and time sync

```
sudo apt update && sudo apt -y upgrade
sudo apt install -y docker.io docker-compose-v2 git chrony
sudo systemctl enable --now docker chrony
sudo usermod -aG docker ubuntu
exit                                    # log out and in again so the docker group applies
```

Time sync (`chrony`) matters: Delta rejects signed requests if the server clock is more than a few seconds off.

Add swap memory (needed on the 1 GB E2.1.Micro, harmless on Ampere). Building the image installs pandas and NumPy, which can run out of memory on 1 GB:

```
sudo fallocate -l 2G /swapfile && sudo chmod 600 /swapfile
sudo mkswap /swapfile && sudo swapon /swapfile
echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab     # keep it after reboots
free -h                                                          # "Swap: 2.0Gi" should show
```

Step 6: Get the code

```
ssh -i ~/Downloads/oracle.key ubuntu@<SERVER_IP>
git clone https://github.com/Sumit211995/delta_bot.git
cd delta_bot
mkdir -p state logs data
```

If the repo is **private**, git asks for a username and password: type `Sumit211995` and paste a GitHub personal access token as the password. Do **not** put the token inside the URL (it would be saved in plain text in `.git/config`). To avoid typing it on every update, use a read-only **deploy key** instead: on the server run `ssh-keygen -t ed25519 -f ~/.ssh/delta_deploy -N ""`, add `~/.ssh/delta_deploy.pub` in GitHub → repo → *Settings → Deploy keys* (leave "write access" off), then clone with

`GIT_SSH_COMMAND="ssh -i ~/.ssh/delta_deploy" git clone git@github.com:Sumit211995/delta_bot.git` and run `git config core.sshCommand "ssh -i ~/.ssh/delta_deploy"` inside the folder.

Step 7: Copy your `.env` safely (run this on your **Mac**, not on the server)

```
scp -i ~/Downloads/oracle.key ~/Documents/Project/delta_bot/.env ubuntu@<SERVER_IP>:~/delta_bot/.env
```

Then on the server: `chmod 600 ~/delta_bot/.env` and check `grep DRY_RUN ~/delta_bot/.env` shows `DRY_RUN=1`.

Optional: continue your laptop's paper records. Stop the laptop bot first (Ctrl+C), then from your Mac:

```
scp -i ~/Downloads/oracle.key -r ~/Documents/Project/delta_bot/state
~/Documents/Project/delta_bot/logs ubuntu@<SERVER_IP>:~/delta_bot/
```

Skip this to start the server's paper records from zero.

Step 8: Start the bot and dashboard

```
cd ~/delta_bot
docker compose up -d --build            # builds the image, starts both containers, restarts them
on reboot/crash
docker compose ps                       # both should be "running"
docker compose logs -f delta-bot        # watch the bot (Ctrl+C only stops watching)
```

You should get **"Bot started … PAPER 4h … also paper-trading 30m, 1h, 4h-trail, 1d-trail"** on Telegram. The first build takes about 3–6 minutes on the free VM.

Step 9: Check the connection from the server

```
docker compose exec delta-bot python bot.py --check      # wallet balance + positions; no orders
docker compose exec delta-bot python bot.py --status     # paper records
```

If `--check` says "IP not whitelisted" or "unauthorized", go back to Step 3. The bot always connects to Delta over **IPv4**, so whitelist the server's public **IPv4** address (`curl -4 ifconfig.me` on the server shows it).

Step 10: Open the dashboard from your laptop

The dashboard listens only on the server itself (`127.0.0.1:8000`), so nobody on the internet can see it. Open a secure tunnel from your Mac:

```
ssh -i ~/Downloads/oracle.key -N -L 8000:localhost:8000 ubuntu@<SERVER_IP>
```

Leave that terminal open and browse to **http://localhost:8000**. Do **not** open port 8000 in Oracle's firewall.

Step 11: Stop the laptop copy

Only **one** bot should run per Delta account. Once the server bot works, stop the one on your laptop (Ctrl+C). You can still run `python dashboard.py` locally to view prices; it reads your laptop's paper records, not the server's.

16.5 Running and maintaining the server

Task	Command (on the server, in `~/delta_bot`)
See if it is running	`docker compose ps`
Watch logs	`docker compose logs -f --tail 100 delta-bot`
Paper results	`docker compose exec delta-bot python bot.py --status`
Update to the latest code	`git pull && docker compose up -d --build` (records and the Binance cache are kept)
Restart	`docker compose restart`
Stop everything	`docker compose down` (live positions keep their exchange SL/TP)
Emergency close (live only)	`docker compose exec delta-bot python bot.py --panic`
Back up records to your Mac	`scp -i ~/Downloads/oracle.key -r ubuntu@<SERVER_IP>:~/delta_bot/{state,logs} ~/delta_backup/`
Change a setting	`nano .env` then `docker compose up -d` (comments on their **own line**, never after a value)
Open the dashboard on phone / laptop	Tailscale app signed in → `https://<server>.<tailnet>.ts.net` (set up once with `sudo tailscale serve --bg http://127.0.0.1:8000`)

How an update flows: change code on your Mac → `git commit` → `git push` → on the server `git pull && docker compose up -d --build`. Never edit code directly on the server; GitHub stays the single source of truth.

- **Auto-restart:** `restart: unless-stopped` in `docker-compose.yml` restarts the bot after a crash or a server reboot.
- **Memory:** `state/`, `logs/` and `data/` are mounted from the server disk, so updates and restarts keep all records and the history cache.
- **Monitoring:** Telegram tells you about starts, stops, trades and repeated errors. If you get no "Bot started" message after an update, check the logs.
- **Security:** keep Ubuntu updated (`sudo apt update && sudo apt -y upgrade` monthly), never open port 8000, only use SSH keys.

16.6 Deploy checklist

- ☐ Code pushed to GitHub, `.env` **not** in the repo
- ☐ Oracle VM running in Mumbai/Hyderabad, public IP noted (reserved if possible)
- ☐ Server IP whitelisted on the Delta API key (Read + Trade, **no Withdrawal**)
- ☐ Docker, Git and chrony installed, 2 GB swap added
- ☐ `.env` copied with `scp`, `chmod 600`, `DRY_RUN=1` (and `state/` + `logs/` if continuing laptop records)
- ☐ `docker compose up -d --build` → both containers running
- ☐ `bot.py --check` works from the server
- ☐ Telegram "Bot started" received
- ☐ Dashboard opens through Tailscale (or the SSH tunnel)

- [ ] Laptop bot stopped (only one bot running)

17. Can I use the trailing-stop strategy? Going live later (not now)

Short answer: yes for paper trading now. Not yet with real money.

1. **Paper:** the "4h trail" record already runs next to the normal 4h record. Let both collect **4–8 weeks** (about 15–30 signals across the 3 coins) and compare them in the dashboard journal.
2. **Why not live yet:** for real orders, the bot would have to move the stop-loss order on Delta after every 4h candle. That needs new live order code and its own testing on testnet. Until it exists, the config refuses the trailing exit when `DRY_RUN=0`.
3. **If you go live before that:** use the normal 4h fixed 1:3. It is also profitable in every test, just less so.
4. **What to expect from trailing:** more wins (about 40%), but many are small; the profit comes from occasional big trends (+5R to +17R) that can be missing for months. Losing streaks of 10 or more still happen.

Live checklist (when you decide)

Do not do this yet. First collect at least 4–8 weeks of paper results. When you decide to go live:

1. Paper results on 4h look in line with the backtest (win rate about 25–35%, losses kept small).
2. Use a **separate Delta sub-account** for the bot, or close manual trades on BTC/ETH/SOL. The bot ignores coins with an open position, and `--panic` closes manual positions too.
3. Fund it with an amount you can afford to lose. With about $27, only ETH fits (`SYMBOLS=ETHUSD` , `BOT_CAPITAL_USD=27` , `RISK_PER_TRADE=0.02`). About ₹15–20k gives the bot room to trade BTC, ETH and SOL at normal risk.
4. Optionally test the real order flow once on **testnet** (`DELTA_ENV=testnet` , testnet keys).
5. Set `DRY_RUN=0` and `LIVE_CONFIRM=I_UNDERSTAND_THE_RISK` , then restart. 30m and 1h stay paper-only automatically.
6. Watch the first live trade: check on Delta that the stop loss and take-profit orders appear.

18. Security rules

- **Never share** the Delta API secret, the Telegram token or the GitHub token. Never paste them in chats or screenshots.
- **Never enable Withdrawal** on the Delta API key. Without it, even a stolen key cannot take money out.
- **Whitelist only your own IPs** (laptop/server). Do not add third-party platform IPs you do not use.
- `.env` stays out of Git (`.gitignore`) and has `chmod 600` on the server.
- The dashboard is read-only and bound to `127.0.0.1` . Reach it on a server only through an SSH tunnel.
- If a secret leaks: delete that API key in Delta immediately, create a new one, update `.env` , restart.

19. Troubleshooting and FAQ

Problem / question	Answer
No trades for days	Normal. 4h gives about 6–8 trades a month across 3 coins; some weeks have none. 30m/1h trade more often.
"Signal skipped: size < 1 contract"	The capital is too small for that coin's stop distance. Expected for SOL with a small wallet.
`--check` says unauthorized / IP not whitelisted	Your current IP (or the server's) is not on the API key's whitelist. Mobile hotspot IPs change often.
"Signature expired"	The computer's clock is off. Turn on automatic time (Mac) or install chrony (server).
Bot says HALTED	Max drawdown was hit, or the protection check failed. Read the reason in `--status`, review, then delete the matching `state/*.json` file to reset.
The dashboard shows nothing	Is `python dashboard.py` running? Refresh the page. Check that internet works.
Will the stop loss work if my laptop is off?	**Live:** yes, SL/TP are stored on Delta. **Paper:** paper trades are only checked while the bot runs; it catches up on the candles it missed when restarted.
Paper records stay empty and the log shows `No such file or directory: '# e.g. ...'`	A `.env` value had a comment after it (`KEY= # note`); Docker passes the comment as the value. Fixed in v1.3; keep comments on their own line.
A paper trade does not show in my Delta app	Correct: paper trades are simulated and only appear on the dashboard and Telegram.
Can the bot trade Indian stocks (Upstox)?	That is a separate project, **upstox_bot** (paper only): index option selling and buying, stock options and share swing trading. See its own guide.
Is the API free?	Yes. Delta and Telegram APIs are free. You pay only normal trading fees (0.05% taker + 18% GST) on real trades.
Can I add more coins?	Test them first (`backtest.py --symbol X`). Of 14 popular coins tested, only BTC, ETH and SOL qualified.

20. Glossary

Term	Meaning
Candle	Open, high, low and close price for one time period (e.g. 4 hours)
Perpetual future	A contract that follows the coin price with no expiry date; you can buy (long) or sell (short)
Contract value	How much coin one contract represents (ETHUSD: 0.01 ETH)
Mark price	The fair price Delta uses for P&L and stop triggers (resists sudden spikes)
EMA	Exponential moving average: an average that weights recent prices more
RSI	Relative Strength Index, 0–100 momentum gauge
ATR	Average True Range: typical movement per candle
Bracket order	An entry order with a stop loss and take profit attached, stored on the exchange
R / R-multiple	Result divided by the amount risked: +3R = won three times the risk; −1R = full stop loss
Profit factor	Total money won ÷ total money lost
Drawdown	The fall from the highest balance to a later low
Backtest	Replaying past data to see how rules would have performed
Walk-forward test	Tune on older data, check on newer unseen data
Paper trading	Simulated trading with live prices and no real money
Slippage	The difference between the expected price and the price you actually get
HMAC-SHA256	The signing method that proves an API request came from your key

This guide describes software for personal use. It is not financial advice. Trading leveraged crypto futures can lose money quickly; only trade what you can afford to lose.